

Python - Access Set Items

Access Set Items

The primary way to access set items is by traversing the set using a loop, such as a **for loop**. By iterating over the set, you can access each element one by one and perform operations on them as needed.

In Python, **sets** are unordered collections of unique elements, and unlike sequences (such as **lists** or **tuples**), sets do not have a positional index for their elements. This means that you cannot access individual elements of a set directly by specifying an index.

Additionally, sets do not have keys associated with their elements, as **dictionaries** do. In a dictionary, each element is paired with a key, allowing you to access the value associated with a specific key. However, sets do not have this key-value pairing.

Therefore, to access the elements of a set we need to use for loop (or, List Comprehension)

Access Set Items Using For Loop

A for loop in Python is a control flow statement used for iterating over a sequence and executing a block of code for each element in the sequence. The loop continues until all elements have been processed.

We can access set items using a for loop by iterating over each element in the set sequentially. Since sets are unordered collections of unique elements, a for loop provides a convenient way to traverse the set and access each element one by one.

Example

In the following example, the for loop iterates over the set "langs", and in each iteration, the variable "lang" is assigned the value of the current element –

```
# Defining a set
langs = {"C", "C++", "Java", "Python"}

# Accessing set items using a for loop
for lang in langs:
    print (lang)
```

It will produce the following output –

```
Python
C
C++
Java
```

Access Set Items Using List Comprehension

List comprehension is an efficient way to create lists in Python. It allows you to generate a new list by applying an expression to each item in an iterable, optionally including a condition to filter elements.

We can access set items using list comprehension by converting the set into a list within the comprehension. This allows you to iterate over the set elements and perform operations on them, similar to using a for loop.

Example

In this example, we are using list comprehension to access set items by iterating over each element of "my_set" –

```
my_set = {1, 2, 3, 4, 5}
# Accessing set items using list comprehension
accessed_items = [item for item in my_set]
print(accessed_items)
```

We get the output as shown below –

```
[1, 2, 3, 4, 5]
```

Access Subset From a Set

Mathematically, a subset is a set that contains only elements that are also contained in another set. In other words, every element of the subset is also an element of the original set. If every element of set **A** is also an element of set **B**, then **A** is considered a subset of **B**, denoted as $A \subseteq B$.

In Python, you can access subsets from a set using set operations or by iterating over the power set (the set of all subsets) and filtering based on specific criteria.

- **Using Set Operations** – You can use built-in set operations such as `issubset()` function to check if one set is a subset of another.
- **Iterating Over Power Set** – Iterate over all possible subsets of the set and filter based on certain criteria to access specific subsets.

Example

Following is the basic example demonstrating how to access subsets from a set –

```
import itertools

# Defining a set
original_set = {1, 2, 3, 4}

# Checking if {1, 2} is a subset of the original set
is_subset = {1, 2}.issubset(original_set)
print("{1, 2} is a subset of the original set:", is_subset)

# Generating all subsets with two elements
subsets_with_two_elements = [set(subset) for subset in
itertools.combinations(original_set, 2)]
print("Subsets with two elements:", subsets_with_two_elements)
```

Following is the output of the above code –

{1, 2} is a subset of the original set: True

Subsets with two elements: [{1, 2}, {1, 3}, {1, 4}, {2, 3}, {2, 4}, {3, 4}]

Checking if Set Item Exists

You can check whether a certain item is available in the set using the **Python's membership operators**, **in** and **not in**.

The **in** operator returns **True** if the specified element is found within the collection, and **False** otherwise. Conversely, the **not in** operator returns **True** if the element is not present in the collection, and **False** if it is.

Example

In the below example, the "in" operator verifies whether the item "Java" exists in the set "langs", and the "not in" operator checks whether the item "SQL" does not exist in the set –

```
# Defining a set
langs = {"C", "C++", "Java", "Python"}

# Checking if an item exists in the set
if "Java" in langs:
    print("Java is present in the set.")
else:
    print("Java is not present in the set.")

# Checking if an item does not exist in the set
if "SQL" not in langs:
    print("SQL is not present in the set.")
else:
    print("SQL is present in the set.")
```

It will produce the following output –

```
Java is present in the set.
SQL is not present in the set.
```

TOP TUTORIALS

[Python Tutorial](#)
[Java Tutorial](#)
[C++ Tutorial](#)
[C Programming Tutorial](#)
[C# Tutorial](#)
[PHP Tutorial](#)
[R Tutorial](#)
[HTML Tutorial](#)
[CSS Tutorial](#)
[JavaScript Tutorial](#)
[SQL Tutorial](#)

TRENDING TECHNOLOGIES

[Cloud Computing Tutorial](#)
[Amazon Web Services Tutorial](#)
[Microsoft Azure Tutorial](#)
[Git Tutorial](#)
[Ethical Hacking Tutorial](#)
[Docker Tutorial](#)
[Kubernetes Tutorial](#)
[DSA Tutorial](#)
[Spring Boot Tutorial](#)
[SDLC Tutorial](#)
[Unix Tutorial](#)

CERTIFICATIONS

[Business Analytics Certification](#)
[Java & Spring Boot Advanced Certification](#)
[Data Science Advanced Certification](#)
[Cloud Computing And DevOps](#)



Chapters ▾

Categories ≡

DevOps Certification

Game Development Certification
Front-End Developer Certification
AWS Certification Training
Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler
Online Python Compiler
Online Go Compiler
Online C Compiler
Online C++ Compiler
Online C# Compiler
Online PHP Compiler
Online MATLAB Compiler
Online Bash Terminal
Online SQL Compiler
Online Html Editor

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.